Modern Programming Principles and Practice
Semester 2
Assessment #1

Module Title: **Modern Programming Principles and Practice**

Assessment Type: Practical Assessment – Individual collaborating

Weighting: **30%**

Maximal Possible Mark: 100 marks

Final Submission Date: AS PER MOODLE

Weekly Submission Dates start: AS PER MOODLE

GITHUB REPO: oop-s2-1-mvc-<student-number> e.g. (oop-s2-1-mvc-1234)

Share to: JohnRowleyDorsetCollege (on github)

Assignment 1: "Community Library Desk"

Focus: EF Core basics + relationships, fake data, searching/filtering, Role creation UI, GitHub Actions CI.

Scenario

A small community library wants a simple internal system to track **Books**, **Members**, and **Loans**. Staff use the app to lend books to members and check what's overdue.

Required tech

- ASP.NET Core MVC
- EF Core + any sql variant (SQL Server, MySQL, Sqlite)
- Identity already set up
- xUnit tests
- GitHub Actions CI workflow (build + test)

Entities (exactly 3 + relationships)

1. **Book**
    - Id, Title, Author, Isbn, Category, IsAvailable
2. **Member**
    - Id, FullName, Email, Phone
3. **Loan**
    - Id, BookId, MemberId, LoanDate, DueDate, ReturnedDate (nullable)

Relationships:

- Book 1—* Loan
- Member 1—* Loan
- Loan belongs to exactly one Book and one Member

Minimum features

A) Admin-only Role Management page (required)

Create a page /Admin/Roles that allows:

- List roles
- Create role (textbox + button)

- Delete role

Rules

- Only Admin can access this page (server-side enforcement).
- Seed an Admin user + Admin role so the page is accessible from day one.

B) CRUD + simple workflows

- Books: list + create + edit + delete
- Members: list + create + edit + delete
- Loans:
    - Create a loan (choose member + choose available book)
    - Mark returned (set ReturnedDate)
    - Prevent lending a book that is already on an active loan (ReturnedDate null)

C) Fake/seed data

On first run (or via migration/seed):

- 20 books
- 10 members
- 15 loans (some returned, some active, some overdue)

Use **Bogus** (recommended).

D) Searching + filtering (mandatory, on one page)

On **Books index page** implement:

- Search by Title or Author (contains)
- Filter by Category (dropdown)
- Filter by Availability (All / Available / On loan)
- Sorting optional (Title A–Z)

This must be done with EF Core query composition (IQueryable), not in-memory filtering.

E) CI workflow (mandatory)

- .github/workflows/ci.yml runs restore/build/test on push + PR to main.

Minimum tests (xUnit)

At least **5 tests**, for example:

- Cannot create a loan for a book already on an active loan
- Returned loan makes book available again
- Book search returns expected matches
- Overdue logic: DueDate < Today and ReturnedDate is null
- Role page authorization (optional if they can test policies; otherwise controller action guards)

Marking

- EF Core model + migrations + relationships: 30
- Fake data seeding: 16
- Books search/filter page: 20
- Loans workflow rules: 14
- Admin Role Management page: 10
- CI workflow passes on GitHub: 10

Marking Rubric

Community Library Desk System

Total: 100 Marks

Criterion 1: EF Core Model & Relationships (30 marks)

Level	Description	Marks
Excellent	Correct 3-entity design (Book, Member, Loan). Proper foreign keys and navigation properties. Relationships enforced correctly. Migrations included and database builds cleanly. Business rules (e.g., no duplicate active loan) correctly implemented.	26–30
Good	Relationships mostly correct. Minor configuration or structural issues. Database builds successfully.	18–25
Satisfactory	Basic persistence works but relational design is weak or partially incorrect.	10–17
Poor	Major relational errors, missing migrations, or broken persistence.	0–9

Criterion 2: Fake Data / Seeding (16 marks)

Level	Description	Marks
Excellent	Meaningful and realistic seed data created (books, members, loans). Includes active, returned, and overdue scenarios.	14–16
Good	Seed data present with reasonable variety.	10–13
Satisfactory	Minimal or simplistic seed data.	5–9
Poor	No meaningful seed data.	0–4

Criterion 3: Search & Filtering Implementation (20 marks)

Level	Description	Marks
Excellent	Search by title/author implemented correctly using EF query composition (IQueryable). Filtering by category and availability works reliably. No in-memory filtering. Clean UI integration.	17–20
Good	Search and filtering mostly correct but minor inefficiencies or logic gaps.	12–16
Satisfactory	Basic search OR filtering implemented but incomplete or partially functional.	6–11
Poor	No working search/filter functionality.	0–5

Criterion 4: Loan Workflow & Business Rules (14 marks)

Level	Description	Marks
Excellent	Prevents duplicate active loans. Returned books become available again. Overdue logic works correctly and consistently.	12–14
Good	Most business rules work but minor logic gaps.	8–11
Satisfactory	Basic loan creation works but lacks robust rule enforcement.	4–7
Poor	Loan workflow unreliable or incorrect.	0–3

Criterion 5: Admin Role Management Page (10 marks)

Level	Description	Marks
Excellent	Admin-only role management page implemented. Roles can be listed and created. Server-side authorization correctly enforced.	9–10
Good	Role creation works but minor authorization or usability gaps.	6–8
Satisfactory	Page exists but incomplete or weakly secured.	3–5
Poor	No meaningful role management functionality.	0–2

Criterion 6: GitHub Repository & CI Workflow (10 marks)

Level	Description	Marks
Excellent	Clean structure (src/tests). README complete. GitHub Actions workflow builds in Release and runs tests successfully on main branch.	9–10
Good	CI present but minor issues or unclear documentation.	6–8
Satisfactory	Repository usable but incomplete CI or inconsistent structure.	3–5
Poor	No CI workflow or failing pipeline on main.	0–2